

Assignment-10.4

Shaik Sabina

Batch-05

2303A51303

TASK-01

PROMPT

Review the given Python code, identify syntax, indentation, variable naming, and function call errors.

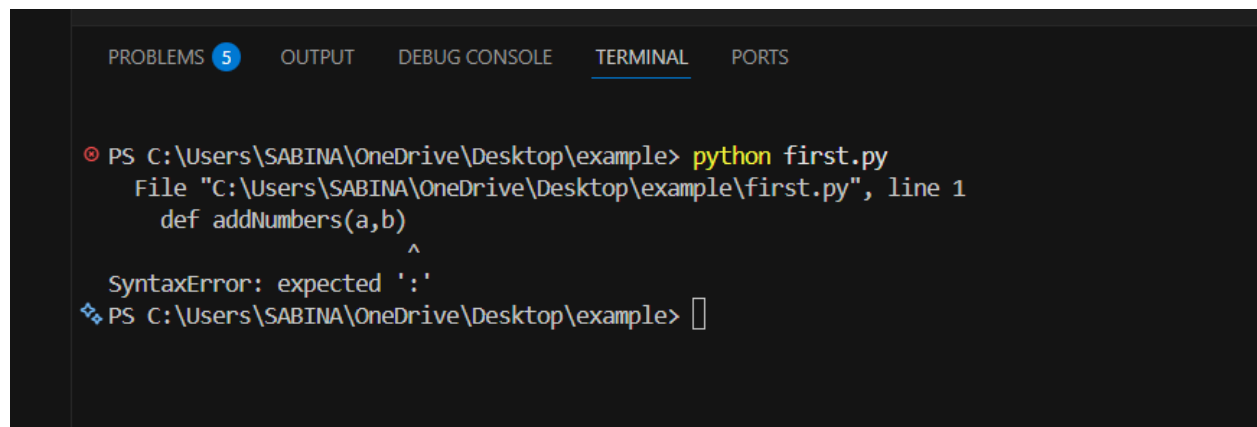
Correct the code and explain each fix.

WRONG CODE

```
def addNumbers(a,b)
result=a+b
print("Sum is",reslt)

addnumbers(5,10)
```

OUTPUT

A screenshot of a terminal window with a dark background. At the top, there are tabs: 'PROBLEMS' with a blue circle containing the number '5', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is underlined), and 'PORTS'. The terminal shows a command prompt 'PS C:\Users\SABINA\OneDrive\Desktop\example>' followed by the command 'python first.py'. Below this, it shows the file path 'File "C:\Users\SABINA\OneDrive\Desktop\example\first.py", line 1' and the code snippet 'def addNumbers(a,b)'. A caret '^' is placed under the closing parenthesis of the function definition. Below the caret, the error message 'SyntaxError: expected ':'' is displayed. The prompt then shows the command 'PS C:\Users\SABINA\OneDrive\Desktop\example>' followed by a cursor.

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\SABINA\OneDrive\Desktop\example> python first.py
File "C:\Users\SABINA\OneDrive\Desktop\example\first.py", line 1
    def addNumbers(a,b)
                      ^
SyntaxError: expected ':'
PS C:\Users\SABINA\OneDrive\Desktop\example> 
```

CODE REVIEW

- Missing colon (:) in function definition
- Indentation is incorrect
- Variable name `reslt` is misspelled
- Function call name does not match definition
- Code does not follow Python naming conventions

CORRECTED CODE

```
def add_numbers(a, b):  
    result = a + b  
    print("Sum is", result)  
  
add_numbers(5, 10)
```

OUTPUT

```
SyntaxError: expected ':'  
PS C:\Users\SABINA\OneDrive\Desktop\example> python first.py  
Sum is 15  
PS C:\Users\SABINA\OneDrive\Desktop\example> 
```

CODE REVIEW

Syntax errors are fixed with proper colon and indentation

Variable names are corrected and meaningful

Function naming follows snake_case convention

Function call matches definition

Code is clean and executable

DESCRIPTION

This task demonstrates AI-assisted syntax and quality review.
All structural and naming issues were identified and corrected.
The final code follows Python standards and executes correctly.
AI helps improve code reliability before deployment.

TASK-02

PROMPT

Analyze the following code for performance issues
and refactor it to improve time complexity.
Explain the improvements.

WRONG CODE

```
def find_duplicates(nums):  
    duplicates = []  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[i] == nums[j] and nums[i] not in duplicates:  
                duplicates.append(nums[i])  
    return duplicates  
  
print(find_duplicates([1,2,3,2,4,1,5]))
```

OUTPUT

```
PS C:\Users\SABINA\OneDrive\Desktop\example> python first.py  
Sum is 15  
PS C:\Users\SABINA\OneDrive\Desktop\example> python second.py  
[1, 2]  
PS C:\Users\SABINA\OneDrive\Desktop\example> 
```

CODE REVIEW

Uses nested loops → $O(n^2)$ time complexity

Becomes very slow for large datasets

Repeated comparisons waste computation time

Not optimized for performance-critical systems

CORRECTED CODE

```
def find_duplicates(nums):  
    seen = set()  
    duplicates = set()  
  
    for num in nums:  
        if num in seen:  
            duplicates.add(num)  
        else:  
            seen.add(num)  
  
    return list(duplicates)  
  
print(find_duplicates([1,2,3,2,4,1,5]))
```

OUTPUT

```
PS C:\Users\SABINA\OneDrive\Desktop\example> python second.py  
[1, 2]  
PS C:\Users\SABINA\OneDrive\Desktop\example>
```

CODE REVIEW

Uses sets for constant-time lookup

Reduces time complexity to $O(n)$

Improves performance significantly

Code is cleaner and scalable

DESCRIPTION

This task highlights AI-assisted performance optimization. Inefficient nested loops were replaced with hash-based logic. Execution speed improves drastically for large datasets. AI helps identify and fix performance bottlenecks.

TASK-03

PROMPT

Refactor the given Python function to improve readability, naming, formatting, and documentation. Apply PEP 8 standards.

WRONG CODE

```
def f(x,y):  
    z=x+y  
    return z
```

OUTPUT

```
[1, 2]  
PS C:\Users\SABINA\OneDrive\Desktop\example> python third.py  
PS C:\Users\SABINA\OneDrive\Desktop\example> 
```

CODE REVIEW

Function name is unclear

Variables have no meaningful names

Poor indentation

No documentation

Hard to maintain long-term

CORRECTED CODE

```
def add_numbers(first_number, second_number):  
    """  
    Adds two numbers and returns the result.  
    """  
    result = first_number + second_number  
    return result  
  
print(add_numbers(5, 10))
```

OUTPUT

```
● PS C:\Users\SABINA\OneDrive\Desktop\example> python third.py  
● PS C:\Users\SABINA\OneDrive\Desktop\example> python third.py  
15  
❖ PS C:\Users\SABINA\OneDrive\Desktop\example> 
```

CODE REVIEW

- Function and variable names are descriptive
- Proper indentation and formatting applied
- Docstring explains function purpose
- Code is maintainable and readable

DESCRIPTION

This task focuses on readability and maintainability.
AI-assisted refactoring improves clarity and structure.
Following PEP 8 standards ensures consistency.
Well-documented code is easier to manage in teams.

TASK-04

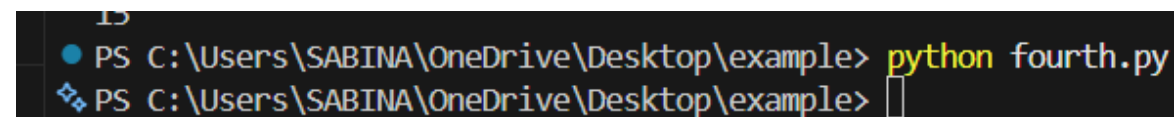
PROMPT

Review the given Python database code for security issues.
Refactor it using safe SQL practices and add error handling.

WRONG CODE

```
def get_user(username):  
    query = "SELECT * FROM users WHERE name = '" + username + "'"   
    cursor.execute(query)  
    return cursor.fetchone()
```

OUTPUT

A terminal window showing a command prompt. The first line shows a blue dot icon followed by the command 'python fourth.py'. The second line shows a blue square icon followed by the prompt 'PS C:\Users\SABINA\OneDrive\Desktop\example>' and a cursor.

```
PS C:\Users\SABINA\OneDrive\Desktop\example> python fourth.py  
PS C:\Users\SABINA\OneDrive\Desktop\example> █
```

CODE REVIEW

Vulnerable to SQL injection attacks

No input validation

No exception handling

Unsafe for production deployment

CORRECTED CODE

```
def get_user(username, cursor):  
    try:  
        query = "SELECT * FROM users WHERE name = %s"  
        cursor.execute(query, (username,))  
        return cursor.fetchone()  
    except Exception as e:  
        print("Error:", e)  
        return None
```

OUTPUT

```
● PS C:\Users\SABINA\OneDrive\Desktop\example> python fourth.py
● PS C:\Users\SABINA\OneDrive\Desktop\example> python fourth.py
❖ PS C:\Users\SABINA\OneDrive\Desktop\example> 
```

CODE REVIEW

Uses parameterized SQL queries

Prevents SQL injection

Exception handling added

Code is secure and production-ready

DESCRIPTION

This task demonstrates secure coding practices.

AI helps detect vulnerabilities and refactor unsafe logic.

Proper error handling improves reliability.

Security is ensured before deployment.

TASK-05

PROMPT

Generate an automated code review report

for the given poorly written Python script.

Identify quality issues and suggest improvements.

WRONG CODE

```
def c(a,b):return a+b
```

OUTPUT


```
PS C:\Users\SABINA\OneDrive\Desktop\example> python fourth.py
PS C:\Users\SABINA\OneDrive\Desktop\example> python fourth.py
PS C:\Users\SABINA\OneDrive\Desktop\example> python fifth.py
```

CODE REVIEW

Issues Identified:

- Poor readability
- Unclear function name
- No documentation
- No formatting standards
- Low maintainability

Risk Areas:

- Difficult for team collaboration
- Error-prone in larger systems

CORRECTED CODE

```
def calculate_sum(num1, num2):
    """
    Returns the sum of two numbers.
    """
    return num1 + num2
```

OUTPUT

```
PS C:\Users\SABINA\OneDrive\Desktop\example> python fifth.py
PS C:\Users\SABINA\OneDrive\Desktop\example> python fifth.py
❖ PS C:\Users\SABINA\OneDrive\Desktop\example> █
```

CODE REVIEW

Code is readable and self-explanatory

Documentation added

Naming conventions improved

Maintainability increased

DESCRIPTION

This assignment demonstrates AI-assisted code review beyond code generation.

AI identifies syntax, performance, readability, and security issues.

Refactoring improves quality, safety, and maintainability.

Automated AI reviews help teams maintain high coding standards